

IAM_App – Documentación Técnica (BD + Backend API)

Equipo Desarrollo TallerApp

20 de agosto de 2025

Índice

1. RP-00 → Introducción y visión general	2
2. RP-10 → Base de Datos (modelo y reglas)	3
2.1. RP-10.1 Estados y convenciones	3
2.2. RP-10.2 Catálogo de alertas (creadas)	3
2.3. RP-10.3 Consideraciones de diseño en SPs	4
3. RP-20 → Stored Procedures relevantes (resumen funcional)	5
3.1. RP-20.1 Tarifas	5
3.2. RP-20.2 Proformas	6
4. RP-30 → Backend API (Controllers, BLL, DAL)	9
4.1. RP-30.1 DTO destacado: DTO_SolicitudDeBusqueda	9
4.2. RP-30.2 Controllers (rutas y propósito)	9
4.3. RP-30.3 BLL (Business Logic Layer)	9
4.4. RP-30.4 DAL (Data Access Layer): parámetros y mapeo	10
4.5. RP-30.5 Seguridad y manejo de errores	10
5. RP-40 → <i>Placeholder</i> Frontend (próximo)	11

1. RP-00 → Introducción y visión general

Este documento consolida la implementación realizada para la gestión de **Tarifario, Proformas e Ítems de Proformas** en el sistema. Se cubren dos capas principales:

- **Base de Datos (BD):** diseño de procedimientos almacenados (SPs), normalización de entradas, reglas de negocio cercanas a los datos (filtros, estados, sumatorias), y catálogo de alertas para mensajes consistentes.
- **Backend API (ASP.NET Core):** controladores (Controllers), lógica de negocio (BLL) y acceso a datos (DAL), utilizando DTOs fuertemente tipados y autorización por roles.

Principios de diseño:

1. **Separación de responsabilidades:** Controllers → BLL → DAL → BD.
2. **Búsqueda flexible con un único término:** se unificó la búsqueda mediante un campo `Term` que acepta texto o números y permite coincidencias en múltiples campos.
3. **Estados coherentes y borrado lógico:** siempre se excluyen los registros marcados como “Eliminados” en listados y búsquedas, pero se pueden actualizar explícitamente a dicho estado.
4. **Mensajes estandarizados:** mediante `UTIL.TBL_ALERTAS` para que Backend y Front interpreten resultados uniformemente.

DTO clave creado `DTO_SolicitudDeBusqueda` permite **pasar un término libre** (`Term`) y **encapsular el negocio** (`DTO_Negocio`) para **acotar** resultados multi-tenant sin cambiar contratos a futuro.

Listing 1: `DTO _SolicitudDeBusqueda` (motivo y uso)

```
namespace DTO
{
    public class DTO_SolicitudDeBusqueda
    {
        public string Term { get; set; } = string.Empty;
        public DTO_Negocio? Negocio { get; set; } = null;
    }
}
```

Razón: incluir `DTO_Negocio` (y no solo `ID_Negocio`) permite escalar criterios sin romper el contrato si mañana se requieren más atributos del negocio en la búsqueda.

2. RP-10 → Base de Datos (modelo y reglas)

2.1. RP-10.1 Estados y convenciones

- **Tarifas:** 23 = Activo, 24 = Inactivo, 25 = Eliminado. Los listados/búsquedas excluyen 25.
- **Proformas:** 26 = Borrador, 27 = Aprobada, 28 = Anulada, 31 = Eliminado. Los listados/búsquedas excluyen 31.
- **Ítems de proforma:** 29 = Activo (solo estos suman al total).

2.2. RP-10.2 Catálogo de alertas (creadas)

Las siguientes alertas fueron insertadas en UTIL.TBL_ALERTAS para estandarizar mensajes:

COD_ALERTA	Nombre	Mensaje	Tipo
B035	Buscar Tarifario	Búsqueda de tarifas realizada correctamente	I
B036	Crear Tarifario	Tarifa creada correctamente	I
B037	Actualizar Tarifario	Tarifa actualizada correctamente	I
B038	Eliminar Tarifario	Tarifa eliminada correctamente	I
B039	Crear Proforma	Proforma creada correctamente	I
B040	Agregar Proforma Item	Ítem agregado a proforma correctamente	I
B041	Eliminar Proforma Item	Ítem eliminado de proforma correctamente	I
B042	Aprobar Proforma	Proforma aprobada correctamente	I
B043	Actualizar Proforma Item	Ítem de proforma actualizado correctamente	I
B044	Registrar Proforma	Consulta de proforma realizada correctamente	I
B045	Actualizar Proforma	Proforma actualizada correctamente	I
B046	Registrar Tarifario	Tarifas obtenidas correctamente	I
B047	Registrar Proforma	Proformas obtenidas correctamente	I
B048	Registrar Proforma Item	Ítems de proforma obtenidos correctamente	I
B049	Buscar Proforma	Búsqueda de proformas realizada correctamente	I

SQL de inserción (referencia)

```
INSERT INTO [UTIL].[TBL_ALERTAS] ([COD_ALERTA], [Nombre], [Mensaje], [Tipo]) VALUES
('B035', N'Buscar Tarifario', N'Bsqueda de tarifas realizada correctamente', 'I'),
('B036', N'Crear Tarifario', N'Tarifa creada correctamente', 'I'),
('B037', N'Actualizar Tarifario', N'Tarifa actualizada correctamente', 'I'),
('B038', N'Eliminar Tarifario', N'Tarifa eliminada correctamente', 'I'),
('B039', N'Crear Proforma', N'Proforma creada correctamente', 'I'),
('B040', N'Agregar Proforma Item', N'tem agregado a proforma correctamente', 'I'),
('B041', N'Eliminar Proforma Item', N'tem eliminado de proforma correctamente', 'I'),
('B042', N'Aprobar Proforma', N'Proforma aprobada correctamente', 'I'),
('B043', N'Actualizar Proforma Item', N'tem de proforma actualizado correctamente', 'I'),
('B044', N'Registrar Proforma', N'Consulta de proforma realizada correctamente', 'I'),
('B045', N'Actualizar Proforma', N'Proforma actualizada correctamente', 'I'),
('B046', N'Registrar Tarifario', N'Tarifas obtenidas correctamente', 'I'),
('B047', N'Registrar Proforma', N'Proformas obtenidas correctamente', 'I'),
('B048', N'Registrar Proforma Item', N'tems de proforma obtenidos correctamente', 'I'),
('B049', N'Buscar Proforma', N'Bsqueda de proformas realizada correctamente', 'I');
```

2.3. RP-10.3 Consideraciones de diseño en SPs

- **Normalización de entradas:** uso de LTRIM/RTRIM y ISNULL para evitar falsos positivos.
- **Ignorar nulos en UPDATE:** se usa ISNULL(@Param, Columna), para no sobrescribir si el parámetro no llegó.
- **Búsqueda con término único (@Term):** si es numérico, intenta match exacto y “contiene” (via CONVERT(...)LIKE) para precios o totales.
- **Exclusión de eliminados:** las consultas de listado/búsqueda siempre excluyen estados “Eliminado”.

3. RP-20 → Stored Procedures relevantes (resumen funcional)

3.1. RP-20.1 Tarifas

SP_ObtenerTarifas (por negocio) Devuelve tarifas de un negocio. Por defecto excluye eliminadas; puede listar activas e inactivas.

- **Parámetros:** @ID_Negocio INT; (opcional: @SoloActivas BIT si se activa en el futuro).
- **Estados:** 23, 24 (excluye 25).

SP_buscarTarifas (término único @Term) Búsqueda “inteligente” por **NombreTarifa**, **DescripcionTarifa**, **EstadoNombre** y **PrecioTarifa** (exacto y “contiene”). Siempre excluye eliminados (25).

Listing 2: CORE.SP_{buscarTarifas}(versincon@Termysineliminados)

```
ALTER PROCEDURE CORE.SP_buscarTarifas
    @ID_Negocio INT,
    @Term NVARCHAR(100) = N''
AS
BEGIN
    SET NOCOUNT ON;
    SET @Term = LTRIM(RTRIM(ISNULL(@Term, N'')));
    DECLARE @TermDec DECIMAL(16,3) = TRY_CONVERT(DECIMAL(16,3), @Term);

    SELECT T.ID_Tarifa, T.ID_Negocio, T.ID_Estado,
           T.NombreTarifa, T.DescripcionTarifa, T.PrecioTarifa,
           T.FechaCreacion, T.FechaModificacion,
           E.Nombre AS EstadoNombre
    FROM CORE.TBL_TARIFAS T
    LEFT JOIN UTIL.TBL_ESTADOS E ON E.ID_Estado = T.ID_Estado
    WHERE T.ID_Negocio = @ID_Negocio
           AND T.ID_Estado <> 25
           AND (
               @Term = N'' OR
               T.NombreTarifa LIKE N'%' + @Term + N'%' OR
               T.DescripcionTarifa LIKE N'%' + @Term + N'%' OR
               E.Nombre LIKE N'%' + @Term + N'%' OR
               (@TermDec IS NOT NULL AND (
                   T.PrecioTarifa = @TermDec OR
                   CONVERT(NVARCHAR(50), T.PrecioTarifa) LIKE N'%' + @Term + N'%'
               ))
           )
    ORDER BY T.NombreTarifa;

    IF @@ROWCOUNT = 0
        SELECT COD_ALERTA, Nombre, Mensaje, Tipo
        FROM UTIL.TBL_ALERTAS WHERE COD_ALERTA = 'B035';
END
```

SP_actualizarTarifa (update parcial) Solo actualiza campos enviados; si un campo no llega, NO se pisa en la tabla. Incluye ISNULL en las columnas y permite cambiar a Eliminado (25) cuando se envía @ID_Estado.

Listing 3: CORE.SP_{actualizarTarifa}(updateparcialconISNULL)

```
ALTER PROCEDURE CORE.SP_actualizarTarifa
```

```

@ID_Tarifa INT,
@ID_Estado INT = NULL,
@NombreTarifa VARCHAR(100) = NULL,
@DescripcionTarifa NVARCHAR(255) = NULL,
@PrecioTarifa DECIMAL(16,3) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE CORE.TBL_TARIFAS
    SET ID_Estado = ISNULL(@ID_Estado, ID_Estado),
        NombreTarifa = ISNULL(@NombreTarifa, NombreTarifa),
        DescripcionTarifa = ISNULL(@DescripcionTarifa, DescripcionTarifa),
        PrecioTarifa = ISNULL(@PrecioTarifa, PrecioTarifa),
        FechaModificacion = GETDATE()
    WHERE ID_Tarifa = @ID_Tarifa;

    SELECT T.*, E.Nombre AS EstadoNombre
    FROM CORE.TBL_TARIFAS T
    LEFT JOIN UTIL.TBL_ESTADOS E ON E.ID_Estado = T.ID_Estado
    WHERE T.ID_Tarifa = @ID_Tarifa;

    SELECT COD_ALERTA, Nombre, Mensaje, Tipo
    FROM UTIL.TBL_ALERTAS WHERE COD_ALERTA = 'B037';
END

```

3.2. RP-20.2 Proformas

SP_ObtenerProformas (por negocio, filtros opcionales) Devuelve proformas de un negocio, excluyendo Eliminadas (31), y con **TotalCalculado** a partir de ítems Activos (29).

Listing 4: CORE.SP_ObtenerProformas(resumen)

```

ALTER PROCEDURE CORE.SP_ObtenerProformas
@ID_Negocio INT,
@ID_Cliente INT = NULL,
@ID_Estado INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT P.*,
        CAST(ISNULL((
            SELECT SUM(CAST(I.PrecioItemProforma AS DECIMAL(16,3)) *
                CAST(I.CantidadItemProforma AS DECIMAL(16,3)))
            FROM CORE.TBL_PROFORMAS_ITEMS I
            WHERE I.ID_Proforma = P.ID_Proforma
                AND I.ID_Estado = 29
            ),0) AS DECIMAL(16,3)) AS TotalCalculado,
        E.Nombre AS EstadoNombre
    FROM CORE.TBL_PROFORMAS P
    LEFT JOIN UTIL.TBL_ESTADOS E ON E.ID_Estado = P.ID_Estado
    WHERE P.ID_Negocio = @ID_Negocio
        AND ( @ID_Cliente IS NULL OR P.ID_Cliente = @ID_Cliente )
        AND ( @ID_Estado IS NULL OR P.ID_Estado = @ID_Estado )
        AND P.ID_Estado <> 31
    ORDER BY P.ID_Proforma DESC;

```

```

SELECT COD_ALERTA,Nombre,Mensaje,Tipo
FROM UTIL.TBL_ALERTAS WHERE COD_ALERTA='B047';
END

```

SP_buscarProformas (término único @Term) Búsqueda flexible por observación, **nombre de estado**, **nombre de cliente** (join a clientes) y **total** (numérico exacto y “contiene”). Excluye Eliminadas (31).

Listing 5: CORE.SP_{buscarProformas}(versincon@Term)

```

ALTER PROCEDURE CORE.SP_buscarProformas
    @ID_Negocio INT,
    @Term NVARCHAR(100) = N'',
AS
BEGIN
    SET NOCOUNT ON;
    SET @Term = LTRIM(RTRIM(ISNULL(@Term, N'')));
    DECLARE @TermDec DECIMAL(16,3) = TRY_CONVERT(DECIMAL(16,3), @Term);

    SELECT P.ID_Proforma, P.ID_Negocio, P.ID_Cliente, P.ID_Estado,
        P.FechaProforma, P.FechaVencimiento, P.ObservacionProforma, P.
            FechaModificacion,
        CAST(ISNULL((
            SELECT SUM(CAST(I.PrecioItemProforma AS DECIMAL(16,3)) *
                CAST(I.CantidadItemProforma AS DECIMAL(16,3)))
            FROM CORE.TBL_PROFORMAS_ITEMS I
            WHERE I.ID_Proforma = P.ID_Proforma AND I.ID_Estado = 29
        ),0) AS DECIMAL(16,3)) AS TotalCalculado,
        E.Nombre AS EstadoNombre,
        C.NombreCliente, C.ApellidoCliente
    FROM CORE.TBL_PROFORMAS P
    LEFT JOIN UTIL.TBL_ESTADOS E ON E.ID_Estado = P.ID_Estado
    LEFT JOIN CORE.TBL_CLIENTES C ON C.ID_Cliente = P.ID_Cliente
    WHERE P.ID_Negocio = @ID_Negocio
        AND P.ID_Estado <> 31
        AND (
            @Term = N'' OR
            P.ObservacionProforma LIKE N'%'+@Term+N'% ' OR
            E.Nombre LIKE N'%'+@Term+N'% ' OR
            (C.NombreCliente LIKE N'%'+@Term+N'% ' OR
            C.ApellidoCliente LIKE N'%'+@Term+N'% ') OR
            (@TermDec IS NOT NULL AND (
                EXISTS (
                    SELECT 1
                    FROM CORE.TBL_PROFORMAS_ITEMS I
                    WHERE I.ID_Proforma = P.ID_Proforma AND I.ID_Estado=29
                    GROUP BY I.ID_Proforma
                    HAVING SUM(CAST(I.PrecioItemProforma AS DECIMAL(16,3)) *
                        CAST(I.CantidadItemProforma AS DECIMAL(16,3))) = @TermDec
                ) OR
                CONVERT(NVARCHAR(50),
                    CAST(ISNULL((
                        SELECT SUM(CAST(I.PrecioItemProforma AS DECIMAL(16,3)) *
                            CAST(I.CantidadItemProforma AS DECIMAL(16,3)))
                        FROM CORE.TBL_PROFORMAS_ITEMS I
                        WHERE I.ID_Proforma = P.ID_Proforma AND I.ID_Estado = 29
                    ),0) AS DECIMAL(16,3))) LIKE N'%'+@Term+N'% '
                ))
            )
        )

```

```

)
ORDER BY P.ID_Proforma DESC;

IF @@ROWCOUNT = 0
    SELECT COD_ALERTA,Nombre,Mensaje,Tipo
    FROM UTIL.TBL_ALERTAS WHERE COD_ALERTA='B049';
END

```

SP_actualizarProforma (update parcial) Actualiza cliente/estado/observación/vencimiento **solo si** se envían; caso contrario conserva valores actuales.

Listing 6: CORE.SP_{actualizarProforma}(updateparcialconISNULL)

```

ALTER PROCEDURE CORE.SP_actualizarProforma
    @ID_Proforma INT,
    @ID_Cliente INT = NULL,
    @ID_Estado INT = NULL,
    @ObservacionProforma NVARCHAR(255) = NULL,
    @FechaVencimiento DATETIME = NULL
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE CORE.TBL_PROFORMAS
    SET ID_Cliente = ISNULL(@ID_Cliente, ID_Cliente),
        ID_Estado = ISNULL(@ID_Estado, ID_Estado),
        ObservacionProforma = ISNULL(@ObservacionProforma, ObservacionProforma),
        FechaVencimiento = ISNULL(@FechaVencimiento, FechaVencimiento),
        FechaModificacion = GETDATE()
    WHERE ID_Proforma = @ID_Proforma;

    SELECT P.*, E.Nombre AS EstadoNombre
    FROM CORE.TBL_PROFORMAS P
    LEFT JOIN UTIL.TBL_ESTADOS E ON E.ID_Estado = P.ID_Estado
    WHERE P.ID_Proforma = @ID_Proforma;

    SELECT COD_ALERTA,Nombre,Mensaje,Tipo
    FROM UTIL.TBL_ALERTAS WHERE COD_ALERTA='B045';
END

```

Nota sobre ítems Los SPs SP_registrarItemsProforma, SP_actualizarItemsProforma y SP_ObtenerItemsProforma siguen el patrón: validaciones básicas, sumatoria coherente (solo estado 29) y retorno de alerta (B040, B043, B048).

4. RP-30 → Backend API (Controllers, BLL, DAL)

4.1. RP-30.1 DTO destacado: DTO_SolicitudDeBusqueda

```
public class DTO_SolicitudDeBusqueda
{
    public string Term { get; set; } = string.Empty;
    public DTO_Negocio? Negocio { get; set; } = null;
}
```

Motivación: unificar búsqueda con un solo término (*type-ahead*: “Dav”, “borrador”, “200”) y acotar por negocio para multi-tenant. Evita cambiar contratos cuando mañana se requiera más metadata del negocio.

4.2. RP-30.2 Controllers (rutas y propósito)

Todos los endpoints están protegidos con `[Authorize(Roles="1")]` y retornan JSON.

ProformaController

- **POST** `/api/Proforma/registrarProforma` — Crea proforma.
- **POST** `/api/Proforma/actualizarProforma` — Update parcial (no pisa nulos).
- **POST** `/api/Proforma/obtenerProformas` — Lista por negocio (+ *filtros opcionales*).
- **POST** `/api/Proforma/buscarProformas` — Búsqueda flexible vía `Term`.

ItemsProformaController

- **POST** `/api/ItemsProforma/registrarItemsProforma` — Alta ítem.
- **POST** `/api/ItemsProforma/actualizarItemsProforma` — Update parcial.
- **POST** `/api/ItemsProforma/obtenerItemsProforma` — Lista ítems por proforma.

TarifarioController

- **POST** `/api/Tarifario/buscarTarifas` — Type-ahead con `Term`.
- **POST** `/api/Tarifario/obtenerTarifas` — Lista por negocio.
- **POST** `/api/Tarifario/registrarTarifa` — Alta tarifa.
- **POST** `/api/Tarifario/actualizarTarifa` — Update parcial con precio opcional.

4.3. RP-30.3 BLL (Business Logic Layer)

La BLL se mantiene **delgada** para no duplicar la lógica de los SPs. Actúa como *fachada*: coordina entradas y devuelve `DTO_Respuesta` de forma uniforme. Permite crecer con validaciones adicionales sin romper contratos.

4.4. RP-30.4 DAL (Data Access Layer): parámetros y mapeo

- Uso de SqlCommand con **parámetros tipados**, enviando DBNull.Value cuando un campo no se envía para que el SP lo ignore con ISNULL.
- **Decimales**: se definen Precision=16, Scale=3 cuando corresponde (p.ej. precios).
- **No enviar precio** en update si no se desea cambiar: *no agregar el parámetro* o enviarlo como NULL para conservar el valor.
- Mapeo seguro desde SqlDataReader con UTL_DBHelper.ReadNullSafe*.

Ejemplos de requests (Postman) Buscar tarifas (type-ahead):

```
POST /api/Tarifario/buscarTarifas
{
  "term": "cambio",
  "negocio": {"ID_Negocio": 1002}
}
```

Buscar proformas (por observación/estado/cliente/total):

```
POST /api/Proforma/buscarProformas
{
  "term": "borrador",
  "negocio": {"ID_Negocio": 1002}
}
```

Actualizar tarifa (solo estado):

```
POST /api/Tarifario/actualizarTarifa
{
  "ID_Tarifa": 3,
  "Estado": {"ID_Estado": 25} // Eliminado
}
```

Actualizar proforma (observación + cliente):

```
POST /api/Proforma/actualizarProforma
{
  "ID_Proforma": 4,
  "ID_Cliente": 2,
  "ObservacionProforma": "Se actualiza"
}
```

4.5. RP-30.5 Seguridad y manejo de errores

- **Autorización** con [Authorize(Roles="1")] en endpoints críticos.
- **Mensajes de alerta** (catálogo UTIL.TBL_ALERTAS) retornados por los SPs y propagados en DTO_Respuesta.
- **Errores no controlados**: capturados por UTL_ManejoError.

5. RP-40 → *Placeholder* Frontend (próximo)

Esta sección se completará más adelante. Lineamientos ya definidos para mantener coherencia con el código existente del usuario:

- **Select de búsqueda (*type-ahead*)** unificado: el input envía `Term` hacia los endpoints de búsqueda.
- **Consistencia con componentes existentes** (ejemplo del usuario):

Listing 7: Patrón de componente a respetar (ejemplo del usuario)

```
/**
 * Estructura base de componentes React a respetar:
 * type Props = {} | interface Props { ... }
 * export const NombreComponente = (props: Props) => (<>...</>)
 *
 * Los "custom renderers" deben seguir el patrón establecido por el autor,
 * p.ej. AsyncClientSelect para selección asíncrona.
 */
```

Se respetará el patrón de *custom renderers* tipo:

- `type Props = {}`
- `export const NombreComponente = (props: Props) =>(<>...</>)`
- `AsyncClientSelect` y similares para selects asíncronos.
- Mantener nomenclatura, documentación en línea y coherencia con el estilo actual.

Ejemplos concretos de UI (formularios, grillas, filtros) se agregarán cuando iniciemos la etapa Frontend.